

1. 3-Schema ANSI/SPARC Architecture

Core Concept

Objective: Physical database se user applications ko decouple karna.

- 1. External Level (View Schema):** Multiple tailored views for end users. Unrelated tables hide hote hain (e.g., Student portal hides teacher salary).
- 2. Conceptual Level (Logical Schema):** Pure database structure ka global blueprint (entities, attributes, relationships, integrity constraints). Hardware independent.
- 3. Internal Level (Physical Schema):** Data physically hard disk par kaise stored hai (B+ Trees, hashing, clustering, file pointers, block allocation).

2. Data Independence

Exam Favourite

Ek schema level ko modify karne par uske higher level ko re-write na karna pade.

- Logical Data Independence:** Conceptual schema modify karne par External schema & user queries untouched rehti hain. (*Very Difficult to achieve*). Example: Adding column `DOB` to table does not break existing views.
- Physical Data Independence:** Internal schema (disk indexing, hashing, sorting) modify karne par Conceptual/External untouched rehti hain. (*Easily achieved*). Example: Changing Heap file to B+ Tree.

3. Traditional File System vs Modern DBMS

10-Marker Must

Evaluation Parameter	Traditional File System (OS Files)	Database Management System (DBMS)
Data Redundancy	Har department duplicate files maintain karta hai; immense disk waste.	Centralized schema; normalization through minimum redundancy.
Data Inconsistency	Ek file update hui, doosri purani reh gayi → Phantom data mismatch.	Single point updates & cascade rules; 100% mutual consistency.
Atomicity Guarantee	No rollback. Mid-transfer system crash leaves money deducted & lost.	WAL (Write-Ahead Logging) & 2-Phase Commit ensure All-or-Nothing.
Concurrent Access	File-level OS lock; reader-writer stalls or race conditions (lost update).	2PL & multi-version timestamping guarantee Serializability.
Security & Abstraction	Operating system coarse file permissions (rwx); no column masking.	Fine-grained column/row level SQL privileges (<code>GRANT SELECT</code>).

4. Database User Categories & DBA Roles

Viva & Theory

- Naive / Parametric:** Form GUI users (ATM, Railway booking, cashier).
- Application Programmers:** DML embedded code likhne wale (Java/C++ devs).
- Sophisticated Users:** Complex ad-hoc SQL / Analytical queries (Data Analysts).
- Specialized Users:** Non-relational apps (GIS spatial data, AI models).
- DBA (Database Administrator) Duties:** Schema definition, Storage structure & access path tuning, Granting authorization (DCL), Periodic backup & crash recovery.

5. DBMS Languages Taxonomy

Gate / AKTU

- DDL (Data Definition):** Schema create/alter. Auto-committed! Commands: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
- DML (Data Manipulation):** Tuples modify/retrieve.
 - Procedural (Low-level):* User specifies **WHAT** data and **HOW** to get it (Relational Algebra).
 - Declarative (High-level):* User specifies only **WHAT** data is needed (SQL, Relational Calculus). Commands: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- DCL (Control):** Permissions. `GRANT`, `REVOKE`.
- TCL (Transaction):** Integrity. `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

6. DBMS Tier Architectures

Architecture

- 1-Tier:** User UI, Client logic, aur Database ek hi physical machine par resident (Local SQLite, MS Access). Zero network overhead.
- 2-Tier:** Client PC par UI + App logic run hoti hai; Database remote server par hota hai via ODBC/JDBC. Fat client bottleneck.
- 3-Tier:** Client (Web Browser) → Application Server (Business Logic) → Database Server (Data Storage). Scalable, secure & enterprise standard.

7. Data Models Classification

Foundations

- 1. Object-Based Logical Models:** High level of abstraction. E-R (Entity-Relationship) Model and Object-Oriented Model.
- 2. Record-Based Logical Models:** Fixed format records.
 - Relational Model:* 2D Tables (Relations) connected by Foreign Keys (Codd, 1970).
 - Network Model:* Directed Graph (Arbitrary Many-to-Many via pointers).
 - Hierarchical Model:* Strict Tree structure (1:N parent-child, IMS).
- 3. Physical Data Models:** Frame memory, Unifying models.

1. Attributes Taxonomy & SymbolsER Elements Simple / Atomic: Cannot be broken further (Age, RollNo). Composite: Hierarchy of sub-parts (Name → First, Mid, Last). Single-Valued: Single item per tuple (Aadhar_No, DOB). Multivalued: Multiple items per entity (Phone_No, Skills). Symbol: Double Ellipse. Derived: Computed dynamically (Age = Current_Date - DOB). Symbol: Dashed Ellipse. Key Attribute: Primary identifier. Symbol: Underlined Text.	2. Cardinality & Participation ConstraintsMapping Logic Cardinality Ratios: 1:1 (One-to-One): Citizen ↔ Passport. 1:N (One-to-Many): Department → Employees. M:N (Many-to-Many): Students ↔ Courses. Participation Constraints: Total Participation (Double Line): Har entity relationship me mandatory appear hoti hai (e.g., Har Employee ka Dept hona mandatory). Partial Participation (Single Line): Kuch entities relation ke bina exist kar sakti hain (e.g., Har Professor Dean nahi hota).
3. Weak Entity Sets & Identifying RelationshipsCrucial Rule Definition: Aisa entity set jiske paas apna Primary Key form karne ke liye sufficient attributes na hon. Yeh identifying Owner Entity par depend karta hai. Weak Entity Symbol: Double Rectangle. Identifying Relationship: Double Diamond (Always 1:N from Owner to Weak, Total participation on Weak side). Partial Key / Discriminator: Dashed Underline attribute. Weak Entity PK = Owner PK + Partial Key (Discriminator) Real-World Canonical Example: - Owner: EMPLOYEE(Emp_ID, Name, Salary) - Weak: DEPENDENT(Dep_Name, Relation, Age) [Partial key: Dep_Name] - Resulting PK: {Emp_ID, Dep_Name} (Composite Primary Key).	
4. Relational Keys Master Hierarchy & Super Key MathFormula & GATE Super Key (SK): Any set of attributes jo relation ke har tuple ko uniquely identify kare. (May have redundant attributes). Candidate Key (CK): Minimal Super Key. Koi bhi proper subset super key nahi hona chahiye! Primary Key (PK): Database designer dwara choose kiya gaya single candidate key. CANNOT BE NULL! Alternate Key (AK): Candidate keys jo Primary Key choose hone se bache rahe (AK = CK - PK). Foreign Key (FK): Current table ka attribute jo kisi parent relation ke Primary Key ko refer karta hai. Compound / Composite Key: Key made of 2 or more attributes. Super Key Calculation Formula: If Relation has n attributes and single Candidate Key of size k: Total SKs = $2^{(n - k)}$. For multiple Candidate Keys: Apply Principle of Inclusion-Exclusion: $SK(A \cup B) = 2^{(n - A)} + 2^{(n - B)} - 2^{(n - A \cup B)}$.	
5. Extended ER (EER) Modeling FeaturesAdvanced ER Core Abstractions: Generalization: Bottom-up approach. Multiple lower entities ki common properties extract karke superclass banana (e.g., Car & Truck → Vehicle). Specialization: Top-down approach. Superclass ko specific distinguishing attributes ke basis par subclasses me split karna (e.g., Employee → Developer, QA). Aggregation: Relationship-as-Entity abstract karna taaki ternary relationship ko binary me clean model kiya ja sake. EER Constraints: Disjointness Constraint: <i>Disjoint ('d')</i>: Entity maximum 1 subclass ki member ho sakti hai (Mutually exclusive). <i>Overlapping ('o')</i>: Entity ek saath multiple subclasses me exist kar sakti hai. Completeness Constraint: <i>Total Specialization (Double Line)</i>: Har superclass entity kisi na kisi subclass me belong karegi. <i>Partial Specialization (Single Line)</i>: Entity bina kisi subclass me gaye superclass me reh sakti hai.	

1. The 7 ER-to-Relational Mapping Rules & Table Minimization Matrix

AKTU Top Scoring

ER Construct	Conversion / Mapping Rule	Min Tables	Resulting Relational Schema Structure
Strong Entity	Create dedicated table. Flatten composite attributes into atomic columns.	1 Table	STUDENT(<u>RollNo</u> , FName, LName, City, Marks)
Weak Entity	Table contains weak attributes + Owner PK as Foreign Key. Composite PK!	1 Table	DEPENDENT(<u>Emp_ID (FK)</u> , <u>Dep_Name</u> , Relation, Age)
1:1 (Both Total)	Merge both entity sets and relationship into a SINGLE unified table!	1 Table	CITIZEN_PASSPORT(<u>Aadhar_No</u> , Name, Pass_No, IssueDate)
1:1 (One Partial)	Maintain 2 tables. Place PK of Partial entity as Foreign Key on Total side.	2 Tables	EMP(<u>Emp_ID</u> , Name) & DEPT(<u>Dept_ID</u> , DName, Mgr_ID(FK))
1:N Relationship	NO new table! Put Primary Key of '1' side as Foreign Key into 'N' side table.	2 Tables	DEPT(<u>Dept_ID</u>) & EMP(<u>Emp_ID</u> , Name, Dept_ID(FK))
M:N Relationship	Create separate Join Table! PK = Composite union of both participating PKs.	3 Tables	STU(<u>Roll</u>), CRS(<u>CID</u>), ENROLL(<u>Roll(FK)</u> , <u>CID(FK)</u> , Grade)
Multivalued Attr	Create separate table. PK = (Entity PK + Multivalued Attribute).	+1 Table	STUDENT_PHONE(<u>RollNo(FK)</u> , <u>Phone_Number</u>)

2. Relational Data Model & The 4 Integrity Constraints

Rules

Domain Constraint:

Har attribute value uske defined valid atomic set me hona chahiye (e.g., $Age > 0$, $Marks \in [0, 100]$).

Key Constraint:

Relation ke koi bhi do distinct tuples identical Primary/Candidate Key share nahi kar sakte.

Entity Integrity:

Primary Key attribute value **KABHI BHI NULL NAHI** ho sakti! (Kyunki NULL identify nahi kar sakta).

Referential Integrity:

Foreign Key value ya to parent relation ke existing valid Primary Key se match karegi, ya fir **NULL** hogi. Violated on Delete/Update → Actions: CASCADE, SET NULL, RESTRICT.

3. Relational Algebra Operations Master Summary

Formal Operators

Operation	Symbol	Type	Behavior & Key Property	Example Syntax
Selection	$\sigma_p(R)$	Horizontal	Rows filter karta hai based on predicate condition p . Degree same, Cardinality $\leq R $.	$\sigma_{Salary > 50000}(EMPLOYEE)$
Projection	$\pi_A(R)$	Vertical	Columns filter karta hai. Automatically removes duplicate rows! Degree = $ A $.	$\pi_{Name, Salary}(EMPLOYEE)$
Cartesian Product	$R \times S$	Cross	Combines all tuples. Degree = $deg(R) + deg(S)$; Cardinality = $ R \times S $.	$STUDENT \times COURSE$
Natural Join	$R \bowtie S$	Join	Common attribute names par Cartesian product followed by equality selection and projection.	$EMP \bowtie DEPT$
Outer Joins	$\&LeftFullJoin;$, $\&LeftThreetimes;$, $\&RightThreetimes;$	Preserve	Unmatched dangling tuples ko retain karte hain padded with NULLs (Left, Right, Full).	$EMP \&LeftThreetimes; DEPT$
Division Operator	$R \div S$	Universal	Solves " FOR ALL / EVERY " queries. (Find students who took ALL math courses).	$\pi_{Roll, CID}(ENROLL) \div \pi_{CID}(MATH_CRS)$

4. Relational Calculus & Codd's Theorem

Theoretical Foundation

Tuple Relational Calculus (TRC):

Non-procedural. Focuses on filtering tuples: $\{ t \mid P(t) \}$. Where t is a tuple variable and $P(t)$ is a first-order formula using $\exists$, $\forall$, $\wedge$, $\vee$, $\neg$.

Domain Relational Calculus (DRC):

Focuses on filtering individual attribute domain variables: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$.

CODD'S THEOREM (Relational Completeness): Relational Algebra = Safe TRC = Safe DRC

A query language is **Relationally Complete** if it can express any query expressible in basic Relational Algebra. SQL is built upon this foundation!